CSC 3320: System-Level Programming

Chapter 2: UNIX Utilities for Non-programmers – Part III and Chapter 1: Introduction to C

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Regular Expression

A pattern describing a certain amount of text.

A "match" is the piece of text that pattern was found to correspond to by the *regex* processing software.

For example,

Note: You might need to use –P in place of –E

Pattern	Will Match
/\d{1,2}\/\d{1,2}\/\d{4}/	Date (e.g. 9/5/2016)
/\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3} /	IP Address (e.g. 255.255.0.0)

```
grep -E --color=auto '\d{1,2}/\d{1,2}/\d{4}' regex_practice.txt
```

Characters

Any character on the keyboard (except newline character '\n')

Literals

- The characters represent themselves within a regex

Special characters

- The characters not represent themselves within a regex
- E.g. **$** matches the end of a line only.

In a regex, the literal characters can be treated as *words*, while the special characters can be treated as *grammar*.

The most basic regex consists of a single literal character, e.g.: **a**

Delimiters

Special characters used to mark the beginning and end of a regex.

Any characters can be used as a delimiter.

But most often, people use forward slash '/'

- E.g. /\d{1,2}\/\d{1,2}\/\d{4}/

Delimiters are not used with **grep** family of utilities (we need quotes at the end of regrex)

Special Characters

The metacharacters with special meaning are

Extended

\ [ ] ^ $. * | ? + ( )

Basic

The dash (-) can only be considered as a metacharacter within the square brackets to indicate a range.

- E.g. **[0-9]** matches the digits through 0 to 9

The brace {} can only be considered as a metacharacter when it is part of repetition operator

- E.g. **(cat) {1,3}** matches the strings containing one to three "cat"

Special Characters

To match any single character, use .

To match any of the single characters in a range, use []

- E.g. [a-z], [2-5], [abc]
- Other metacharacters lose their special meanings inside square bracket

To convert a metacharacter into a literal, use backslash \

- E.g. \[, \/

Special Characters

Regex	Matches	Examples
/.nd/	All strings with any character preceding **nd**	Say all your prayers and then, Oh you sleepy young heads dream of wonderful things, And you will be swimming there too.
[A-D]	Member of the character class A through D	Beautiful mermaids will swim through the sea, And you will be swimming there too.
\.	A period	And you will be swimming there too.
a.	A string with prefix a	Say all your prayers and then, Oh you sleepy young heads dream of wonderful things, Beautiful mermaids will swim through the sea,

Special Characters

$ matches the end of a line only

- E.g. >$

^ matches the beginning of a line only

- E.g. ^ID
- Note: **[^abc]** means any characters excluding a, b and c

Regex	Matches	Examples
/a.$/	A string with prefix **a** ends a line	Beautiful mermaids will swim through the sea,
/^.nd/	All strings with any character preceding **nd** and at beginning of a line	And you will be swimming there too.

Special Characters

? matches **zero or one** occurrence of the single preceding character.

- E.g. a?, abc?

* matches **zero or more** occurrences of the character that precedes it.

- E.g. a*, abc*

- + matches **one or more** occurrences of the single preceding character.
  - E.g. a+, abc+

Special Characters

( ) group the matched string by regex inside it

- E.g. (abc)* v.s. abc* (will empty string "" match ?)

| matches either expression, acts like an "or " operator

- E.g. (a|b) , (a|bc), a|bc*

Regex	Matches	Examples
/off|will/	**off** or **will** may appear	And you **will** be swimming there too.
(ab){1,2}c	**ababc** **abc**	ababc Aababd This is abc

Character Classes

Character class	Matches
\s	White space
\S	Not white space
\d	Digit
\D	Not digit
\w	Word
\W	Not word
\x	Hexadecimal digit
\O	Octal digit

Global Regular Expression Print (grep)

Example: Find all lines containing the word "error" in a log file:

- grep "error" /var/log/syslog

Example: Find lines that start with the word "error":

- grep "^error" /var/log/syslog

Example: Find all lines that do **not** contain the word "success":

- grep -v "success" file.txt

Practices

Write regular expressions for following patterns.

- Phone number with the format ###-###-####
- Date with the format MM/DD/YYYY
- Passwords: strings with at least one upper case letter, one lower case letter, and one digit

Choose all matched strings of the given extended regular expression.

- (1) a(bc)+de
  A) abcde B) ade C) abcbcde D) abc
- (2) [a-z\s]*hello
  A) Othello B) hello C) 2hello D) say hello

Basic Features of C

Chapter 1 – Introduction to C
Chapter 2 – C Fundamentals
Chapter 3 – Formatted Input/output
Chapter 4 – Expression
Chapter 5 – Selection Statement
Chapter 6 – Loops
Chapter 7 – Basic Types
Chapter 8 – Arrays
Chapter 9 – Functions

Advanced Features of C

Chapter 11– Pointers

Chapter 12 – Pointers and Arrays

Chapter 13 – Strings

Chapter 14 – The Preprocessor

Chapter 16 – Structures, Union and Enumerations

Chapter 17 – Advanced Uses of Pointers

Chapter 15 – Writing Large Programs

Origins of C

C is a by-product of UNIX, developed at Bell Laboratories by Ken Thompson, Dennis Ritchie, and others.

Thompson designed a small language named B.

B was based on BCPL, a systems programming language developed in the mid-1960s.

Origins of C

By 1971, Ritchie began to develop an extended version of B.

He called his language NB ("New B") at first.

As the language began to diverge more from B, he changed its name to C.

The language was stable enough by 1973 that UNIX could be rewritten in C.

C-Based Languages

C++ includes all the features of C, but adds classes and other features to support object-oriented programming.

Java is based on C++ and therefore inherits many C features.

C# is a more recent language derived from C++ and Java.

Python is written in C (CPython)